ET4351 Lab 2
Synthesis and Design Constraints

Created by: Chang Gao (chang.gao@tudelft.nl), May 2023
Updated by: Chang Gao, Feb 2025

1 Introduction

In this laboratory exercise, you will gain a comprehensive understanding of the following topics:

- Digital circuit synthesis in `Cadence Genus`.
- Post-synthesis Simulation.

2 Tutorial 1: Synthesis with Cadence Genus

Lab 1 focused on a higher-level, abstract representation of circuits known as the Register-Transfer-Level (**RTL**). In RTL design, we concentrate on the flow and transformation of data between registers and the logic controlling those flows without delving into the physical implementation details.

In the next stage of the design process, we will transform the RTL design into a lower-level representation known as a `gate-level netlist`, using the synthesis tool `Cadence Genus`[1]. This netlist is functionally equivalent to the original RTL design but is expressed in terms of specific logic gates and flip-flops rather than abstract data flows. The netlist is typically realized as a comprehensive Verilog file, detailing the instances and interconnections of these so-called standard cells.

Semiconductor foundries, such as Taiwan Semiconductor Manufacturing Company (TSMC) or GlobalFoundries, provide standard cells, for example logic gates and flip-flops. These foundries offer a collection of standard cells, known as the `standard cell library`, containing many gates that can be fabricated in a digital design for a specific technology node.

In the following tutorials, detailed instructions will be provided on using `Cadence Genus` to convert RTL designs into a gate-level netlist. We will guide you to synthesize a 4-bit integer multiplier circuit step by step.

First of all, let's copy the folder of Lab 2 to your home directory and navigate to the `tut/synth` folder:

```
cd ~
cp -r /data/labs/2025/labs/lab2 ~/
cd lab2
source setup.sh
cd tut/synth
```

[1] Cadence Genus: `https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/genus-synthesis-solution.html`

2.1 Adding Constraints

To ensure that your design is synthesized without issues, it is crucial to ensure that our designs meet specific performance requirements. To do this, we use so-called constraints, which are rules that define aspects of the design process, such as timing, power, or area requirements. These constraints guide the synthesis process to ensure the final design meets the desired specifications.

The `Cadence Genus` synthesis tool defines these constraints in a Synopsys Design Constraints (**SDC**) file[2]. The SDC file is a script written in the Tool Command Language (**TCL**).

An SDC file typically includes information such as:

- **Clock definitions** specify the clock signals in the design, their frequencies, and any clock uncertainties. Example commands are: `create_clock`, `set_clock_uncertainty`.
- **Input and output delays** define the maximum and minimum delays for data signals entering and leaving the design. Example commands are: `set_input_delay`, `set_output_delay`.
- **False paths and multicycle paths** are exceptions to the normal timing paths. False paths are timing paths that do not need to be considered during timing analysis, while multicycle paths allow the data to propagate for more than one clock cycle. Example commands are: `set_false_path`, `set_multicycle_path`.
- **Power, area, and other optimization constraints** can guide the synthesis tool to optimize for power, area, or other parameters as per the requirements of the design, of which the syntax of the command can be different across EDA tools.

We have prepared a `mul.sdc` file under `~/lab2/tut/src`, in which we defined 100 ns clock period, 0.25 ns clock uncertainty, and 5 ns input/output delays. Take a moment to inspect the file and try to understand what each line is doing according to what you learned during the lecture.

2.2 Process Design Kit & Other Settings

Before the circuit synthesis process, we must configure specific settings of our synthesis tools. These settings primarily focus on the technology delineated by the Process Design Kit (PDK). Furnished by foundries like TSMC or GlobalFoundries, the PDK is a suite of files that encapsulates extensive details about a particular manufacturing process.

In the context of synthesis, our foremost task is to guide the synthesis tool toward the directories containing the **Standard Cell Libraries** (`.lib` files). A standard cell library is a collection of pre-designed and pre-verified digital logic components, such as logic gates (AND, OR, XOR, etc.), flip-flops, and memory elements. Each cell in the library is fully characterized by the foundry, meaning its performance (timing, power, area) is accurately known for various operating conditions. This characterization is stored in .lib files (also known as Liberty files), which include information about cell functionality, timing, power, and more. Synthesis and timing analysis tools use the `.lib` files to optimize the design and check whether it meets the required timing and power constraints.

We have prepared the `synth_set.tcl` script for you, which could be run in Cadence Genus with the following command:

```
1    genus -legacy_ui -64 -f scripts/synth_set.tcl
```

[2]SDC was initially developed by `Synopsys`, a leading Electronic Design Automation (**EDA**) company, and thus carries the name "Synopsys". However, the SDC format has been widely adopted across the industry and is now considered a standard. It benefits designers as it allows for consistency and compatibility across different tools and stages of the design flow.

The following piece of this script sets up the paths to the Cadence Generic 45nm technology and library files that are needed for synthesis.

```
# Technology files
set TECH_PATH        "/data/Cadence/gpdk045_v60/gsclib045_svt_v4.7/gsclib045_tech"
set TECH_LEF         "${TECH_PATH}/lef/gsclib045_tech.lef"

# Standard cells
set STD_CELLS_PATH   "/data/Cadence/gpdk045_v60/gsclib045_svt_v4.7/gsclib045"
set STD_CELLS_LIB    "${STD_CELLS_PATH}/timing/slow_vdd1v0_basicCells.lib"
set STD_CELLS_LEF    "${STD_CELLS_PATH}/lef/gsclib045_macro.lef"

# SRAMs
set SRAM_PATH        "/data/Cadence/gpdk045_v60/Synopsys_sram"
set SRAM_LIB         "${SRAM_PATH}/saed32sram_ss0p95vn40c.lib"
set SRAM_LEF         "${SRAM_PATH}/saed32sram.lef""
```

- `TECH_PATH` is set to the path where the technology files for the 45nm process are stored.
- `TECH_LEF` is the Library Exchange Format (**LEF**) file for the technology layer. It provides a representation of the physical layout of the technology (layers, vias, etc) and is used during the physical design process. This file is optional for our synthesis flow but will be indispensable for the physical design process in Lab 3.
- `STD_CELLS_PATH` points to the location of the standard cell library, which is a set of pre-designed digital logic gates like `AND`, `OR`, `NOT`, `flip-flops`, etc.
- `STD_CELLS_LIB` points to the `.lib` file for the standard cell library. In this case, the library is the "slow" version at a supply voltage of 1.0V.
- `STD_CELLS_LEF` is the LEF file for the standard cell library.
- `SRAM_PATH` is set to the path where the Static Random Access Memory (**SRAM**) instances are stored.
- `SRAM_LIB` is the `.lib` file for the SRAM instances.
- `SRAM_LEF` is the `.lef` file for the SRAM instances.

2.3 Elaboration

Elaboration is the first stage in the synthesis process, where Hardware Description Language (**HDL**) language code is interpreted and transformed into a generic logic netlist. Read the `SystemVerilog` code and elaborate on the multiplier design by keying the following line in the command line of `Genus`:

```
read_hdl -sv "${INPUT_PATH}/${DESIGN}.sv"
elaborate $DESIGN
read_sdc "${INPUT_PATH}/${DESIGN}.sdc"
syn_generic
```

Once the elaboration phase is complete, the synthesis tool has a detailed understanding of the design. It is ready to proceed with the subsequent steps of the synthesis process, such as optimization and mapping to the target technology library.

2.4 Mapping

Mapping involves translating the technology-independent, generic logic netlist into a technology-dependent netlist comprised of specific standard cells from the library.

At the end of the elaboration phase, the synthesis tool has an RTL netlist that represents the functionality of the design in terms of generic operations such as logical `AND`, `OR`, and `NOT` operations, flip-flops, and memory elements. This netlist is technology-independent, meaning it does not yet reflect the specific capabilities or limitations of the target manufacturing process.

During the mapping stage, the synthesis tool takes this generic netlist and maps it onto the specific standard cells available in the library for the target technology. This complex task involves matching the operations in the RTL netlist with the functionality provided by the standard cells while optimizing for various objectives such as speed, area, and power consumption.

Start the mapping process by:

```
1    syn_map
```

The actual synthesis process is done after this step.

2.5 Export

After synthesis, we want to check whether our design meets the timing, area requirements and extract other useful information.

Extract the area and timing report by:

```
1    report area > ${REPORTS_PATH}/${DESIGN}_area.rpt
```

Extract the timing report of the 10 worst timing paths by:

```
1    report timing -worst 10 > ${REPORTS_PATH}/${DESIGN}_timing.rpt
```

Examine the timing report and search for the slack of the 10 worst timing paths. These values should all be non-negative, indicating that the design has satisfied the timing requirement.

Finally, we must export the outputs including the netlist `mul.struct.v`, `mul.struct.sdc` and `mul.struct.sdf` files.

```
1    change_names -verilog
2    write_hdl ${DESIGN} > ${OUTPUTS_PATH}/${DESIGN}.struct.v
3    write_sdc ${DESIGN} > ${OUTPUTS_PATH}/${DESIGN}.struct.sdc
4    write_sdf -nonegchecks -interconn "interconnect" -delimiter "/" > ${OUTPUTS_PATH}/${
         DESIGN}.struct.sdf
```

- `change_names -verilog` renames all objects in the design to make them compatible with the Verilog naming conventions. It ensures that the names of objects (like nets, cells, ports etc.) do not contain any characters that are not allowed in Verilog.
- `write_hdl` writes out the current state of the design hierarchy in Verilog format to the specified output file.
- `write_sdc` writes out the current design constraints in SDC format to the specified output file.
- `write_sdf` writes out the delay information of the current design in Standard Delay Format (**SDF**). The `-nonegchecks` option skips the generation of negative timing checks. The `-interconn "interconnect"` option specifies that the interconnect delays are to be included in the output. The `-delimiter "/"` option sets the hierarchical delimiter to `"/"`.

2.6 Post-Synthesis Standard Delay Format (SDF)

SDF is an IEEE standard (originally proposed by Cadence in 1990-91) that encodes timing data such as path delays, interconnect delays, and timing checks for digital designs. Most EDA tools use SDF for *back-annotation*, where delays derived from a specific stage (e.g., post-synthesis or post-layout) are fed back into simulations or STA (static timing analysis). SDF may also be used for *forward-annotation* to convey constraints or incremental updates to subsequent tools.

Below is an annotated example from the SDF you just generated using Cadence Genus. We provide explanations for each key element in the SDF header and cell sections.

```
(DELAYFILE
  (SDFVERSION  "OVI 3.0")              // SDF standard version
  (DESIGN      "mul")                  // Top-level design name
  (DATE        "Tue Feb 18 10:18:10 CET 2025")
  (VENDOR      "Cadence, Inc.")
  (PROGRAM     "Genus(TM) Synthesis Solution")
  (VERSION     "21.10-p002_1")
  (DIVIDER     /)                      // Hierarchical divider ("/" or ".")
  (VOLTAGE     ::0.9)                  // Operating voltage(s)
  (PROCESS     "::1.0")                // Process corner factor
  (TEMPERATURE ::125.0)                // Operating temperature
  (TIMESCALE   1ps)                    // Time units for all delay values

  (CELL
    (CELLTYPE "mul")
    (INSTANCE)
    (DELAY
      (ABSOLUTE
        (INTERCONNECT clk c_reg[5]/CK (::0.0) (::0.0))
        (INTERCONNECT clk c_reg[6]/CK (::0.0) (::0.0))
        ...
      )
    )
  )

  (CELL
    (CELLTYPE "MX2X1")
    (INSTANCE g1045__2398)
    (DELAY
      (ABSOLUTE
        (IOPATH A  Y (::282) (::270))   // Delay from input A -> output Y
        (IOPATH B  Y (::244) (::232))   // Delay from input B -> output Y
        (IOPATH S0 Y (::572) (::580))   // Delay from select S0 -> output Y
      )
    )
  )

  (CELL
    (CELLTYPE "DFFRHQX8")
    (INSTANCE c_reg[5])
    (DELAY
      (ABSOLUTE
        (IOPATH RN Q () (::166))         // Asynchronous reset -> Q delay
        (IOPATH CK Q (::369) (::360))    // Clock -> Q delay
      )
    )
    (TIMINGCHECK
      (RECREM (posedge RN) (posedge CK) (::0.0) (::84)) // Recovery/Removal check for RN w.r.t
```

```
          clock
      (SETUPHOLD (negedge D) (posedge CK) (::113) (::0.0))
      (SETUPHOLD (posedge D) (posedge CK) (::169) (::0.0))
    )
  )
  ...
)
```

Header Section Highlights

- **(SDFVERSION "OVI 3.0"):** Identifies the SDF standard version.
- **(DESIGN "mul"):** Specifies the top-level module or design name that this SDF applies to.
- **(DIVIDER /):** Defines the character used for hierarchical path separation.
  - If `DIVIDER` is `/`, then a path might look like: `top_design/block1/block2/AND`
  - If `DIVIDER` is `.`, then the same hierarchy is written as: `top_design.block1.block2.AND`
- **(VOLTAGE 1.2:1.0:0.8) (Illustration):** Sometimes, you will see multiple voltages in the format `(min:typ:max)`, denoting the supply voltages used in **minimum**, **typical**, and **maximum** (alternatively **best**, **nominal**, **worst**) corners, respectively.
- **(TIMESCALE 100 ps) (Illustration):** Specifies that all numeric delay values in the file should be multiplied by 100 ps.
  - For example, if you see: `IOPATH (posedge A) Z (2:3:4) (5:6:7)`, it means the min:typ:max delays become: (0.2ns : 0.3ns : 0.4ns) for **rise** ($0 \rightarrow 1$ transitions) and (0.5ns : 0.6ns : 0.7ns) for **fall** ($1 \rightarrow 0$ transitions) after multiplying by 100 ps.

Min/Typ/Max Corners

Many SDF lines include three values in a colon-separated triple, for example `(IOPATH A Y (1.2:1.5:1.8) (2.1:2.4:2.7))`. These values typically refer to:

- **Minimum (min)**: The fastest scenario (e.g., highest voltage, lowest temperature, best-case process).
- **Typical (typ)**: A nominal scenario (e.g., nominal voltage, mid temperature, typical process).
- **Maximum (max)**: The slowest scenario (lowest voltage, highest temperature, worst-case process).

They are named *min*, *typ*, and *max* because from the perspective of path delay:

- In the *min* corner, transistor speed is highest, so delays are minimal.
- In the *typ* corner, transistor speed is nominal.
- In the *max* corner, transistors are slow, giving the maximum observed delay.

Cell Section Highlights

- **(CELLTYPE "DFFRHQX8"):** Corresponds to the type of cell (e.g., a flip-flop from a standard library).
- **(INSTANCE c_reg[5]):** Indicates the specific instantiation of that cell in your hierarchy.
- **(DELAY (ABSOLUTE ...)) vs. (DELAY (INCREMENT ...)):**
  - *ABSOLUTE* overwrites existing delay information.
  - *INCREMENT* adds to any previously specified delays (e.g., for incremental updates).
- **(IOPATH A Y (X) (Y)):** Shows the delay from an input pin `A` to an output pin `Y`. The two parenthesized values often reflect rise/fall.
- **(INTERCONNECT net1 net2 (X) (Y)):** Describes interconnect (net) delay from `net1` to `net2`.
- **(TIMINGCHECK ...)** encloses setup, hold, recovery, and removal timing constraints to ensure correct sequential circuit operation. Figure 1 illustrates these constraints for a single D-type Flip-Flop.

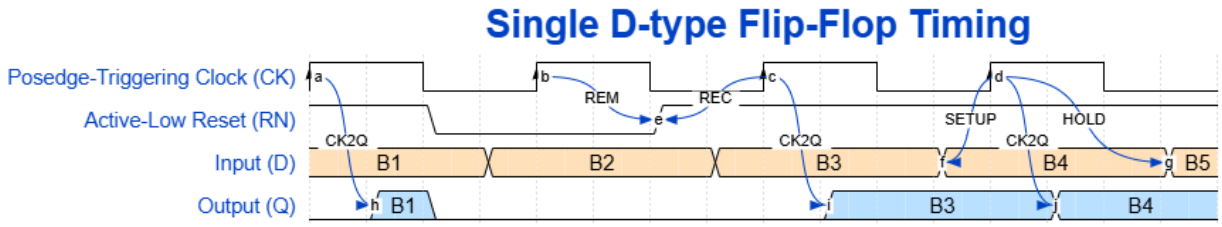


Figure 1: Positive Edge-Triggering D-type Flip-Flop Timing Diagram illustrating Setup, Hold, Recovery, and Removal constraints.

  - *Recovery/Removal*: These checks apply to asynchronous control signals such as `reset` (`RN` in this example where 'N' stands for 'active-low'), ensuring safe synchronization with the clock:
    - **Recovery Time**: The minimum time after `RN` is deasserted (goes high) before the next rising clock edge to prevent metastability. In Figure 1, recovery time is represented between points **e** and the next rising edge of `CK` or point **c**. Example:

```
(RECREM (posedge RN) (posedge CK) (::0.0) (::84))
```

    - **Removal Time**: The minimum time that `RN` must remain active (low) after the clock edge before being released, ensuring proper reset operation. In Figure 1, removal time is between points **b** and **e**, ensuring that reset is held long enough to clear the state.
  - *Setup/Hold*: These checks ensure that the data input (`D`) is stable around the active clock edge (`CK`):
    - **Setup Time**: The minimum time before the active clock edge that `D` must be stable, ensuring correct data sampling. This is shown at point **f** in Figure 1, where `D` must be stable before the rising edge of `CK`. Example:

```
(SETUPHOLD (posedge D) (posedge CK) (::169) (::0.0))
```

This means that D must be valid for at least 169 ps before the rising edge of CK.

* **Hold Time**: The minimum time after the active clock edge that D must remain stable to avoid data corruption. In Figure 1, this is indicated at point **g**, where D must remain stable after the rising edge of CK. At a post-synthesis stage, the physical path delays are still unknown and hold violations cannot be properly evaluated. Thus, the hold delays are set to zero.

Practical Usage of SDF

- *Gate-level Simulation with Back-Annotation:* Simulation tools like Questa Sim[3], or VCS[4] read the SDF to insert realistic delays into the netlist simulation, ensuring timing matches your sign-off assumptions.
- *STA Tools:* PrimeTime[5], Tempus[6], and Innovus[7] (physical design, which you will use in Lab 3) ensure consistent timing checks by reading or writing SDF. This consistency between synthesis, P&R, and final sign-off helps prevent mismatches.
- *Forward-Annotation:* Sometimes SDF timing or net-delay overrides can be forwarded to a subsequent step in the flow, letting each tool adopt the same sets of constraints.

By maintaining consistent SDF files across tools, design teams ensure that all parts of the flow (synthesis, simulation, timing analysis) reference the same delay and constraint data, reducing the risk of timing closure surprises late in the tape-out process.

3 Tutorial 2: Static Timing Analysis

Static Timing Analysis (**STA**) is a critical aspect of the digital design flow, as it validates the timing performance of a circuit by evaluating all possible paths for potential timing violations. Unlike dynamic simulation, which requires specific input stimuli and observes the full behavior of a circuit, STA evaluates timing without the necessity of simulating the circuit's logical operation, making it a more efficient approach.

STA decomposes a design into multiple timing paths and computes the signal propagation delay for each one. It then checks these delays against the timing constraints set for the design, both internally and at the input/output interfaces. The design must be adjusted to ensure proper operation if any violation occurs.

The main elements of a timing path are:

- **Startpoint:** This is the initial point of a timing path where a clock edge launches data or where data must be ready at a specific time. It can be either an input port or a clock pin of a register.

[3] https://eda.sw.siemens.com/en-US/ic/questa/simulation/advanced-simulator/

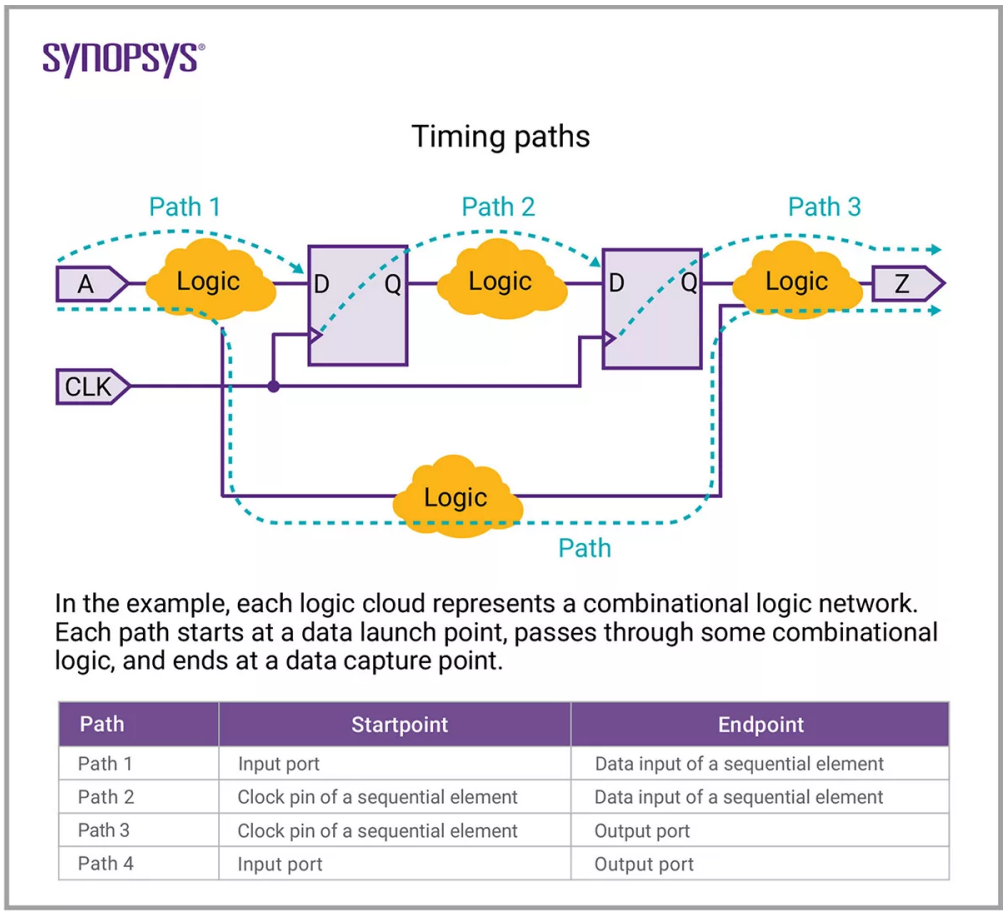
[4] https://www.synopsys.com/verification/simulation/vcs.html

[5] https://www.synopsys.com/implementation-and-signoff/signoff/primetime.html

[6] https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/silicon-signoff/tempus-timing-signoff-solution.html

[7] https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html

- **Combinational logic network:** These elements have no memory or internal state. Combinational logic includes AND, OR, XOR gates, and inverters, but not memory elements like flip-flops, latches, registers, or RAM.
- **Endpoint:** This is the termination of a timing path, where a clock edge captures data or where data must be ready at a specific time. It can be either a data input pin of a register or an output port.



Path	Startpoint	Endpoint
Path 1	Input port	Data input of a sequential element
Path 2	Clock pin of a sequential element	Data input of a sequential element
Path 3	Clock pin of a sequential element	Output port
Path 4	Input port	Output port

Figure 2: The four critical timing paths (Source: `https://www.synopsys.com/glossary/what-is-static-timing-analysis.html`).

Figure 2 shows a simple example where the timing paths are illustrated as lines connecting the startpoints through a combinational logic cloud and ending at the endpoints. The four paths can be named as:

- t_{a2d} (Path 1): From input ports to the data pins of registers.
- t_{ck2d} (Path 2): From clock pins of registers to the data pins of registers.
- t_{ck2z} (Path 3): From clock pins of registers to the output ports.
- t_{a2z} (Path 4): From input ports to the output ports.

A combinational logic cloud may contain multiple paths, and the delays along these paths can vary. STA considers the longest path to calculate the maximum delay (worst-case scenario) and the shortest path for the minimum delay (best-case scenario). This is important because it ensures the design will function correctly under all possible conditions.

You can use the `report timing` command in combination with options like `all_inputs`, `all_outputs`, `all_registers`, to extract timing information of the paths. To specify the pins of registers, use flags such as `-data_pins`, `output_pins`, and `clock_pins`. Here is an example of extracting the timing report of t_{ck2d}.

```
report timing -worst 10 -from [all_registers -clock_pins] -to [all_registers
    -data_pins]
```

4 Tutorial 3: Clock Gating & Power Estimation

4.1 Clock Gating

Clock gating is a prevalent technique in digital circuits to reduce dynamic power consumption. Dynamic power is consumed whenever there is an activity in the circuit, primarily when clock signals toggle. Clock gating mitigates this by adding extra logic (usually using the Integrated Clock Gating (**ICG**) cell[8]; however, it is not the case for us since we are using a generic Process Design Kit (**PDK**) for education purpose) to disable the clock signal to certain parts of the design where it's not needed, preventing unnecessary toggling and thereby conserving power.

Modern synthesis tools often have automated clock gating capabilities, which can intelligently detect opportunities for clock gating during synthesis. These tools can identify idle parts of the circuit and insert clock gating logic automatically, which significantly simplifies the design process. This can lead to substantial power savings, especially in large designs where not all parts of the circuit are active all the time.

It's worth noting that while automated clock gating is a powerful feature, care must still be taken. The addition of clock-gating logic can potentially introduce new timing issues or functional errors. Therefore, even with automatic clock gating, thorough design validation and careful consideration of the impact on the overall system timing and functionality are still critically important.

We have prepared a TCL script `~/lab2/tut/synth/synth_cg.tcl` for you to synthesize the multiplier with clock gating[9]. The script `synth_cg.tcl` calls four sub-scripts, `synth_set.tcl`, `synth_elbcg.tcl`, `synth_map.tcl`, `synth_exp.tcl` for each synthesis step we discussed before. The commands related to clock gating are in `synth_elbcg.tcl`:

```
set_attribute lp_insert_clock_gating true /
set_attribute lp_clock_gating_control_point precontrol /des*/*
set_attribute lp_clock_gating_style latch /des*/*
set_attribute lp_insert_discrete_clock_gating_logic true
```

- `set_attribute lp_insert_clock_gating true /`: enables the automatic insertion of clock gating during synthesis. The `lp` stands for "low power", indicating that this command is part of Genus' low power optimization features.
- `set_attribute lp_clock_gating_control_point precontrol /des*/*`: sets the control point for clock gating to be "precontrol", which means that the enable signal is controlled before

[8] More about ICG cells: `https://teamvlsi.com/2021/08/integrated-clock-gating-icg-cell-in-vlsi.html`
[9] We also provided `~/lab2/tut/synth/synth.tcl` without clock gating for your convenience.

the clock gating cell, typically through a combinational logic function. The `/des*/*` is a wildcard path that applies this setting to all design blocks.

- `set_attribute lp_clock_gating_style latch /des*/*`: This command sets the clock gating style to "latch". This means the clock gating cells will be latch-based.
- `set_attribute lp_insert_discrete_clock_gating_logic true`: This command enables the insertion of discrete clock gating logic during synthesis. This means that instead of using ICG cells, Genus will combine standard cells to achieve clock gating.

Before we proceed, type `exit` in the `Genus` command line to return to the shell. Now, run the synthesis again with clock gating using our provided script:

```
genus -legacy_ui -64 -f scripts/synth_cg.tcl
```

The script will be suspended once the SDC file is read and analyzed. Type `resume` in the terminal to proceed.

Once finished, extract the clock gating report using the following command in the `Genus` command line:

```
report_clock_gating
```

In the report, you should see 16 flip-flops synthesized in the design, and they are 100% clock gated.

Quit `Genus` by typing in:

```
exit
```

5 Tutorial 4: Post-Synthesis Simulation & Power Estimation

5.1 Post-Synthesis (Structural) Simulation

In Lab 1, we did `behavioral simulations` on the 32-bit integer multiplier and `PicoSoC`, which took place before the synthesis process. This type of simulation is done on the initial design, which is written in a high-level hardware description language (in our case,`Verilog` or `SystemVerilog`). Behavioral simulation checks the logic functionality and timing of the design at a high level. The main goal of the behavioral simulation is to verify that the design behaves as expected according to its specification. It doesn't concern itself with the specifics of how the design will be implemented in hardware, such as which specific gates will be used. Instead, it focuses on the overall operation and functionality of the design.

In this tutorial, you will learn how to do `post-synthesis` or `structural simulation`, still using the integer multiplier as an example. `Post-synthesis simulation` is performed after the synthesis process to simulate the behavior of the gate-level netlist. It confirms that the synthesized design behaves as expected, in accordance with the specifications outlined in the original high-level code. This type of simulation is crucial for detecting and rectifying any discrepancies that might have occurred during the synthesis process.

Launch the `post-synthesis simulation` from a `Bash` terminal by:

```
cd ~/lab2/tut/sim_struct
source tb_mul.sh
```

The `post-synthesis simulation` is successful once the outputs match those in your `behavioral simulation`, which could be run using the `tb_mul.sh` script in the directory `~/lab2/tut/sim_behav`.

5.2 Activity Annotation for Post-Synthesis Power Estimation

Activity annotation essentially involves annotating, or assigning, each signal in the design with a value that represents its activity rate - how frequently it switches from a low state to a high state, or vice versa, over a period of time.

This switching activity is typically represented as a Switching Activity Interchange Format (**SAIF**) file[10] or a Value Change Dump (**VCD**) file, which are generated by simulating the design with a set of representative input vectors, usually derived from the expected use-case scenarios of the design.

The importance of activity annotation lies in its direct impact on power estimation. Dynamic power dissipation, which often constitutes a significant portion of the total power dissipation, is directly proportional to the square of the voltage, the capacitance being driven, and the switching activity factor. Thus, having an accurate estimate of the switching activity is crucial to estimate a design's power consumption accurately.

By performing activity annotation, designers can identify which parts of the design switch most frequently, thereby consuming the most power. This information can then be used to guide power optimization strategies, such as clock gating or power gating, to reduce the overall power consumption of the design. Furthermore, accurate power estimation facilitated by proper activity annotation is also crucial for thermal analysis and ensuring the reliability of the design.

Run the post-synthesis simulation again with activity annotation, which will generate a `.vcd` file under `~/lab2/tut/sim_struct/vcd`:

```
mkdir vcd
source tb_mul_vcd.sh
```

Run the synthesis again:

```
cd ~/lab2/tut/synth
genus -legacy_ui -64 -f scripts/synth_cg.tcl
```

Once finished (do not forget the `resume` step), do not exit the `Genus` command line, type in the following command to annotate the multiplier circuit and do the power estimation:

```
read_vcd ../sim_struct/vcd/mul.struct.vcd
report_power
```

The power report shows that the total power of the multiplier circuit is around `1.7 μW`, where 39.8% from registers, 56.2% from combinational logic and 3.9% from clock signals. However, this post-synthesis power estimation should be used only as a reference for refining the design. For more accurate power calculations, we will need to conduct a post-layout power analysis, which will be introduced in Lab 3.

Quit `Genus` by typing in:

```
exit
```

6 Task 1: Synthesize PicoSoC

In this task, you will synthesize the whole `PicoSoC` with a dummy accelerator you simulated in Lab 1 - Task 2.

Ensure you are in the Lab 2 task directory:

[10] Usually used in `Synopsys` tools.

```
cd ~/lab2/task/synth
```

Run the synthesis of `PicoSoC` without clock gating from a `Bash` shell terminal:

```
genus -legacy_ui -64 -f scripts/synth.tcl
```

You can only move on to Task 2 once this task is finished.

7 Task 2: Post-Synthesis Simulation of PicoSoC

Navigate to the `sim_struct` folder:

```

cd ~/lab2/task/sim_struct
```

Run the post-synthesis simulation with:

```
source tb_et4351.sh
```

You are successful if "Hello, World!" is printed on your terminal.

8 Individual Report Questions

Please provide answers to the following questions, as they may be related to potential inquiries during your written examination:

1. In Task 1, please inspect the area and timing reports and write down the chip area with the correct units and the slack of the worst path.
2. In Task 1, change the clock period in `et4351.sdc` to 1 ns (corresponding to 1 GHz clock frequency) and run the synthesis script again. Inspect the change-of-area and timing report. Please write down your observations and explain the changes in area and timing numbers (Check out the lecture slides on timing optimization).
3. Analyze the impact of clock gating on the power efficiency of a PicoSoC by comparing its power consumption with and without clock gating implemented. Please explain how clock gating reduces power consumption in digital integrated circuits. Can it also reduce leakage power?